CD Assingment 2

Name - Rithvik Rajesh Matta
Class - 6H
SRN - PES2UG23CS485

Output

```
pes2ug23cs485@pes2ug23cs485: ~/sem6/CD/assignment2
build.sh  lexer.l  parser.y  test_input.c
pes2ug23cs485@pes2ug23cs485:~/sem6/CD/assignment2$ chmod +x build.sh
./build.sh
Cleaning...
Running Bison...
Running Flex...
Compiling...
Running with test input...
Enter source code (Ctrl+D to end):
Done! AST -> ast_output.txt | IC -> ic_output.txt

=== AST Output ===
=== Abstract Syntax Tree ===

SEQ
  ASSIGN
    ID(x)
    BINOP(+)
      ID(a)
      ID(b)
  SEQ
    IF_ELSE
      COND:
        RELOP(<)
          ID(x)
          ID(y)
      THEN:
        ASSIGN
          ID(z)
          BINOP(+)
            ID(x)
            NUM(1)
      ELSE:
        ASSIGN
          ID(z)
          BINOP(+)
            ID(y)
            NUM(2)
    DO_WHILE
      BODY:
        ASSIGN
          ID(x)
          BINOP(+)
```

```
pes2ug23cs485@pes2ug23cs485: ~/sem6/CD/assignment2

          ID(z)
          BINOP(+)
            ID(x)
            NUM(1)
      ELSE:
        ASSIGN
          ID(z)
          BINOP(+)
            ID(y)
            NUM(2)
    DO_WHILE
      BODY:
        ASSIGN
          ID(x)
          BINOP(+)
            ID(x)
            NUM(1)
      COND:
        RELOP(<)
          ID(x)
          ID(y)

=== Intermediate Code ===
=== Intermediate Code (Quadruples) ===

(+, a, b, t1)
(=, t1, -, x)
(if <, x, y, L1)
(goto, -, -, L2)
L1:
(+, x, 1, t2)
(=, t2, -, z)
(goto, -, -, L3)
L2:
(+, y, 2, t3)
(=, t3, -, z)
L3:
L4:
(+, x, 1, t4)
(=, t4, -, x)
(if <, x, y, L4)
(goto, -, -, L5)
L5:
pes2ug23cs485@pes2ug23cs485:~/sem6/CD/assignment2$
```

Parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* ————————————————————————————————————————————————————————
```

```
  AST Node Types
  ──────────────────────────────────────────────────────────────── */
typedef enum {
    NODE_ASSIGN,
    NODE_BINOP,
    NODE_ID,
    NODE_NUM,
    NODE_IF_ELSE,
    NODE_DO_WHILE,
    NODE_SEQ,       /* sequence of statements */
    NODE_RELOP
} NodeType;

typedef struct ASTNode {
    NodeType type;
    char    *value;          /* operator, id name, number, relop */
    struct ASTNode *left;
    struct ASTNode *right;
    struct ASTNode *extra;   /* used for if-else: condition, then, else */
} ASTNode;

/* ────────────────────────────────────────────────────────────────
  AST helpers
  ──────────────────────────────────────────────────────────────── */
ASTNode *newNode(NodeType t, const char *val,
                 ASTNode *l, ASTNode *r) {
    ASTNode *n = malloc(sizeof(ASTNode));
    n->type  = t;
    n->value = val ? strdup(val) : NULL;
    n->left  = l;
    n->right = r;
    n->extra = NULL;
    return n;
}

/* ────────────────────────────────────────────────────────────────
  AST Printer
  ──────────────────────────────────────────────────────────────── */
void printAST(ASTNode *node, int indent, FILE *out) {
    if (!node) return;
    for (int i = 0; i < indent; i++) fprintf(out, "  ");

    switch (node->type) {
        case NODE_ASSIGN:
            fprintf(out, "ASSIGN\n");
            printAST(node->left,  indent+1, out);
            printAST(node->right, indent+1, out);
            break;
        case NODE_BINOP:
            fprintf(out, "BINOP(%s)\n", node->value);
            printAST(node->left,  indent+1, out);
            printAST(node->right, indent+1, out);
            break;
        case NODE_RELOP:
            fprintf(out, "RELOP(%s)\n", node->value);
            printAST(node->left,  indent+1, out);
            printAST(node->right, indent+1, out);
            break;
        case NODE_ID:
            fprintf(out, "ID(%s)\n", node->value);
            break;
```

```
        case NODE_NUM:
            fprintf(out, "NUM(%s)\n", node->value);
            break;
        case NODE_IF_ELSE:
            fprintf(out, "IF_ELSE\n");
            for (int i = 0; i < indent+1; i++) fprintf(out, "  ");
            fprintf(out, "COND:\n");
            printAST(node->left,  indent+2, out);
            for (int i = 0; i < indent+1; i++) fprintf(out, "  ");
            fprintf(out, "THEN:\n");
            printAST(node->right, indent+2, out);
            for (int i = 0; i < indent+1; i++) fprintf(out, "  ");
            fprintf(out, "ELSE:\n");
            printAST(node->extra, indent+2, out);
            break;
        case NODE_DO_WHILE:
            fprintf(out, "DO_WHILE\n");
            for (int i = 0; i < indent+1; i++) fprintf(out, "  ");
            fprintf(out, "BODY:\n");
            printAST(node->left,  indent+2, out);
            for (int i = 0; i < indent+1; i++) fprintf(out, "  ");
            fprintf(out, "COND:\n");
            printAST(node->right, indent+2, out);
            break;
        case NODE_SEQ:
            fprintf(out, "SEQ\n");
            printAST(node->left,  indent+1, out);
            printAST(node->right, indent+1, out);
            break;
    }
}

/* ────────────────────────────────────────────────────────────────────
   Intermediate Code Generation
   ──────────────────────────────────────────────────────────────────── */
static int tempCount  = 0;
static int labelCount = 0;
static FILE *icOut    = NULL;

char *newTemp() {
    char *t = malloc(16);
    sprintf(t, "t%d", ++tempCount);
    return t;
}

char *newLabel() {
    char *l = malloc(16);
    sprintf(l, "L%d", ++labelCount);
    return l;
}

void emit(const char *op, const char *arg1,
          const char *arg2, const char *result) {
    /* Quadruple format: (op, arg1, arg2, result) */
    fprintf(icOut, "(%s, %s, %s, %s)\n",
            op,
            arg1   ? arg1   : "-",
            arg2   ? arg2   : "-",
            result ? result : "-");
}
```

```
void emitLabel(const char *label) {
    fprintf(icOut, "%s:\n", label);
}

/* Returns the "place" (temp or id/num) of the expression */
char *genExpr(ASTNode *node);

void genCode(ASTNode *node);

char *genExpr(ASTNode *node) {
    if (!node) return strdup("-");

    if (node->type == NODE_ID || node->type == NODE_NUM)
        return strdup(node->value);

    if (node->type == NODE_BINOP) {
        char *l = genExpr(node->left);
        char *r = genExpr(node->right);
        char *t = newTemp();
        emit(node->value, l, r, t);
        free(l); free(r);
        return t;
    }

    if (node->type == NODE_ASSIGN) {
        char *r = genExpr(node->right);
        emit("=", r, "-", node->left->value);
        free(r);
        return strdup(node->left->value);
    }

    return strdup("-");
}

/* Generate IC for a relational condition.
   Emits a conditional jump "if NOT condition goto falseLabel"
   and returns the falseLabel so the caller can place it. */
char *genCondJump(ASTNode *cond, char *trueLabel, char *falseLabel) {
    /* cond is NODE_RELOP */
    char *l = genExpr(cond->left);
    char *r = genExpr(cond->right);

    /* Build "if l rel r goto trueLabel" */
    char op[32];
    sprintf(op, "if %s", cond->value);   /* e.g. "if <" */
    emit(op, l, r, trueLabel);

    /* Unconditional jump to false */
    emit("goto", "-", "-", falseLabel);

    free(l); free(r);
    return falseLabel;
}

void genCode(ASTNode *node) {
    if (!node) return;

    switch (node->type) {

        case NODE_ASSIGN: {
            char *r = genExpr(node->right);
```

```
    emit("=", r, "-", node->left->value);
    free(r);
    break;
}

case NODE_SEQ:
    genCode(node->left);
    genCode(node->right);
    break;

/* if (C) { S1 } else { S2 } S_next
   ------------------------------------------
   evaluate C
   if C goto L_true
   goto L_false
   L_true:
     S1
     goto L_after
   L_false:
     S2
   L_after:
     S_next  (handled by parent SEQ)
*/
case NODE_IF_ELSE: {
    char *lTrue  = newLabel();
    char *lFalse = newLabel();
    char *lAfter = newLabel();

    genCondJump(node->left, lTrue, lFalse);

    emitLabel(lTrue);
    genCode(node->right);          /* then-body */
    emit("goto", "-", "-", lAfter);

    emitLabel(lFalse);
    genCode(node->extra);          /* else-body */

    emitLabel(lAfter);
    break;
}

/* do { S } while (C);
   ---------------------------
   L_start:
     S
     evaluate C
     if C goto L_start
   L_after:
*/
case NODE_DO_WHILE: {
    char *lStart = newLabel();
    char *lAfter = newLabel();

    emitLabel(lStart);
    genCode(node->left);           /* body */

    /* condition jump back */
    char *l = genExpr(node->right->left);
    char *r = genExpr(node->right->right);
    char op[32];
    sprintf(op, "if %s", node->right->value);
```

```
            emit(op, l, r, lStart);
            emit("goto", "-", "-", lAfter);
            emitLabel(lAfter);

            free(l); free(r);
            break;
        }

        case NODE_BINOP:
        case NODE_ID:
        case NODE_NUM:
        case NODE_RELOP:
            /* standalone expression – generate but discard result */
            free(genExpr(node));
            break;
    }
}

/* ————————————————————————————————————————————————————————
   Bison plumbing
   ———————————————————————————————————————————————————————— */
int  yylex(void);
void yyerror(const char *s) { fprintf(stderr, "Parse error: %s\n", s); }

%}

%union {
    char    *str;
    struct ASTNode *node;
}

%token <str> T_ID T_NUM
%token IF ELSE DO WHILE
%token ASSIGN SEMICOLON
%token LPAREN RPAREN LBRACE RBRACE
%token PLUS MINUS MUL DIV
%token <str> LT GT LE GE EQ NE

%type <node> program stmtlist stmt ifstmt dowhilestmt
%type <node> expr term factor condition rel

%left PLUS MINUS
%left MUL DIV

%%

program
    : stmtlist
        {
            FILE *astOut = fopen("ast_output.txt", "w");
            if (astOut) {
                fprintf(astOut, "=== Abstract Syntax Tree ===\n\n");
                printAST($1, 0, astOut);
                fclose(astOut);
            }

            icOut = fopen("ic_output.txt", "w");
            if (icOut) {
                fprintf(icOut, "=== Intermediate Code (Quadruples) ===\n\n");
                genCode($1);
                fclose(icOut);
```

```
        }

        printf("Done! AST -> ast_output.txt | IC -> ic_output.txt\n");
      }
    ;

stmtlist
    : stmt              { $$ = $1; }
    | stmt stmtlist     { $$ = newNode(NODE_SEQ, NULL, $1, $2); }
    ;

stmt
    : T_ID ASSIGN expr SEMICOLON
      { $$ = newNode(NODE_ASSIGN, NULL,
                 newNode(NODE_ID, $1, NULL, NULL), $3); }
    | ifstmt     { $$ = $1; }
    | dowhilestmt { $$ = $1; }
    ;

/* if (C) { S } else { S } S  —— the trailing S is part of stmtlist */
ifstmt
    : IF LPAREN condition RPAREN LBRACE stmtlist RBRACE
      ELSE LBRACE stmtlist RBRACE
      {
        $$ = newNode(NODE_IF_ELSE, NULL, $3, $6);
        $$->extra = $10;
      }
    ;

dowhilestmt
    : DO LBRACE stmtlist RBRACE WHILE LPAREN condition RPAREN SEMICOLON
      { $$ = newNode(NODE_DO_WHILE, NULL, $3, $7); }
    ;

condition
    : T_ID rel T_ID
      {
        $$ = $2;
        $$->left  = newNode(NODE_ID, $1, NULL, NULL);
        $$->right = newNode(NODE_ID, $3, NULL, NULL);
      }
    ;

rel
    : LT  { $$ = newNode(NODE_RELOP, "<",  NULL, NULL); }
    | GT  { $$ = newNode(NODE_RELOP, ">",  NULL, NULL); }
    | LE  { $$ = newNode(NODE_RELOP, "<=", NULL, NULL); }
    | GE  { $$ = newNode(NODE_RELOP, ">=", NULL, NULL); }
    | EQ  { $$ = newNode(NODE_RELOP, "==", NULL, NULL); }
    | NE  { $$ = newNode(NODE_RELOP, "!=", NULL, NULL); }
    ;

expr
    : expr PLUS  term { $$ = newNode(NODE_BINOP, "+", $1, $3); }
    | expr MINUS term { $$ = newNode(NODE_BINOP, "-", $1, $3); }
    | term            { $$ = $1; }
    ;

term
    : term MUL factor { $$ = newNode(NODE_BINOP, "*", $1, $3); }
    | term DIV factor { $$ = newNode(NODE_BINOP, "/", $1, $3); }
```

```
    | factor          { $$ = $1; }
    ;

factor
    : T_ID  { $$ = newNode(NODE_ID,  $1, NULL, NULL); }
    | T_NUM { $$ = newNode(NODE_NUM, $1, NULL, NULL); }
    | LPAREN expr RPAREN { $$ = $2; }
    ;

%%

int main(void) {
    printf("Enter source code (Ctrl+D to end):\n");
    yyparse();
    return 0;
}
```

Lexer.l

```
%{
#include "parser.tab.h"
#include <string.h>
%}

%%

"if"        { return IF; }
"else"      { return ELSE; }
"do"        { return DO; }
"while"     { return WHILE; }
"<="        { return LE; }
">="        { return GE; }
"=="        { return EQ; }
"!="        { return NE; }
"<"         { return LT; }
">"         { return GT; }
"="         { return ASSIGN; }
"+"         { return PLUS; }
"-"         { return MINUS; }
"*"         { return MUL; }
"/"         { return DIV; }
"("         { return LPAREN; }
")"         { return RPAREN; }
"{"         { return LBRACE; }
"}"         { return RBRACE; }
";"         { return SEMICOLON; }

[a-zA-Z_][a-zA-Z0-9_]* {
    yylval.str = strdup(yytext);
    return T_ID;
}

[0-9]+(\.[0-9]+)? {
    yylval.str = strdup(yytext);
    return T_NUM;
```

```
}

[ \t\n\r]+  { /* skip whitespace */ }
.           { /* ignore unknown */ }

%%

int yywrap() { return 1; }
```

Build.sh

```
#!/bin/bash

echo "Cleaning..."
rm -f parser.tab.* lex.yy.c ic_generator ast_output.txt ic_output.txt

echo "Running Bison..."
bison -d parser.y
if [ $? -ne 0 ]; then echo "Bison failed!"; exit 1; fi

echo "Running Flex..."
flex lexer.l
if [ $? -ne 0 ]; then echo "Flex failed!"; exit 1; fi

echo "Compiling..."
gcc parser.tab.c lex.yy.c -o ic_generator
if [ $? -ne 0 ]; then echo "GCC failed!"; exit 1; fi

echo "Running with test input..."
./ic_generator < test_input.c

echo ""
echo "=== AST Output ==="
cat ast_output.txt

echo ""
echo "=== Intermediate Code ==="
cat ic_output.txt
```

Test_input.c

```
x = a + b;
if (x < y) {
    z = x + 1;
} else {
    z = y + 2;
}
do {
    x = x + 1;
} while (x < y);
```